1.4.- Representación de la información.

1.4.1.- Definición de un sistema de numeración.

Un sistema de numeración es un conjunto de símbolos y reglas que se utiliza para la representación de datos numéricos o cantidades.
En los sistemas de numeración existe un elemento característico que define el sistema y que se denomina **base**, siendo ésta el número de símbolos que se utilizan para dicha representación. La representación de una cantidad se efectúa por medio de cadenas de símbolos, cada uno de ellos con un significado que depende de su posición. Por ello, estos sistemas se denominan posicionales.
La notación matemática de la base, para distinguir a cuál de ellas nos estamos refiriendo, consiste en poner al final del número un subíndice formado por un paréntesis abierto a la derecha y el número de la base correspondiente.

NÚMERO EN BASE $B_{(B}$

Ejs:

base 2 → $101001_{(2}$

base 4 → $3102_{(4}$

base 8 → $75021_{(8}$

base 16 → $AB352_{(16}$

base 10 → 89798

NOTA: Si no aparece ningún subíndice, se presupone por defecto base 10.

1.4.2.- Sistemas de numeración posicionales: Uso del TFN.

En primer lugar, hemos de tener en cuenta que los sistemas de numeración que estudiaremos van a ser posicionales, es decir, cada cifra dependerá no sólo de su valor sino de la posición que ocupa.
Ej:

$$672 = 600 + 70 + 2$$

Centenas: 6.

Decenas: 7.

Unidades: 2.

Todos los sistemas de numeración posicionales toman como referencia el punto decimal y tienen una base de numeración a izquierda (positivos) y derecha (negativos) de éste.

$$X_4X_3X_2X_1X_0.X_{-1}X_{-2}X_{-3}{}_{(B}$$

Así:

El TFN es un teorema que relaciona una cantidad expresada **en cualquier sistema de numeración** con la misma cantidad expresada **en el sistema decimal**.

En síntesis, el teorema indica el valor decimal de una cantidad expresada en una base cualquiera a partir de la fórmula:

$$N^o = \sum_{i=-d}^{n} X_i \times B^i$$

donde:

B = Base del sistema de numeración origen

i = Posición respecto al punto.

d = Número de cifras a la derecha del punto.

n = Número de cifras a la izquierda del número menos 1.

X = Cada una de las cifras que componen el número en base B.

Nº = número en base decimal (resultado)

El rango o conjunto de dígitos válidos (X) vendrá determinado por la base, siendo:

Dígitos permitidos = {0..base-1}

1.4.3.- Sistemas de numeración más usuales.

Aunque podemos representar números en la base que queramos, los sistemas de numeración más usuales son:

a) **Sistema Decimal (base 10).** Corresponde al número de símbolos del sistema: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

b) **Sistemas Binario (base 2).** Cada dígito de un número representado en este sistema se denomina bit, que viene de la contracción de binary digit.

c) **Sistemas Octal (base 8) y Hexadecimal (base 16)**={0..9,A,B,C,D,E,F}. Al tratarse de múltiplos del sistema binario, se utilizan en computación para trabajar con mayor comodidad cuando los números a representar son demasiado grandes (recordar que el

sistema octal se forma con grupos de 3 dígitos binarios y el hexadecimal con grupos de 4)

OCTAL	BINARIO
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

HEXADECIMAL	BINARIO
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

1.4.4.- Conversiones.

1.4.4.1.- Conversión de base (b) a base 10.

Aplicamos el TFN explicado en el apartado 1.4.2.

Ejemplo: Convertir el nº $516_{(7}$ a base 10. 7 → 10

Reemplazando los valores correspondientes en la fórmula del TFN:

$516_{(7} = X_2*B^2 + X_1*B^1 + X_0 * B^0 = 5 * 7^2 + 1 * 7^1 + 6 * 7^0 = 5*49 + 1*7 + 6 = 245 + 7 + 6= 258$

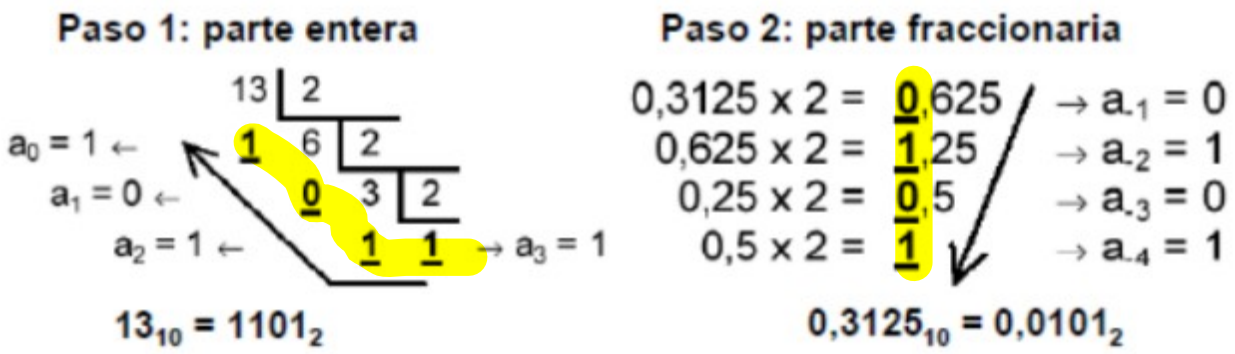
1.4.4.2.- Conversión de base 10 a base (b).

Este tipo de conversión se utiliza para cambiar un número N de base 10 a cualquier otra base (b). Para ello se deben realizar dos pasos por separado:

Paso 1: Convertir la parte entera. Dividimos sucesivamente entre la base b hasta obtener un cociente de 0. La parte entera del número que estamos buscando lo compondrán los restos que se hayan ido obteniendo tomados en orden inverso. Otra opción (ver figura) es dividir hasta que el cociente sea menor que el dividendo, en ese caso habrá que tomar el cociente como bit más significativo. Los bits se toman igualmente en orden inverso.

Paso 2: Convertir la parte fraccionaria. Multiplicamos repetidamente por la base b hasta obtener un cero en la parte fraccionaria o hasta que tengamos el número de cifras requerido. La parte fraccionaria del número buscado lo formarán las partes enteras de los números que se hayan ido obteniendo en cada producto tomadas en ese mismo orden.

Ej: Convertir el nº 13,3125 a binario.



$13{,}3125 = 1101{,}0101_{(2}$

Para comprobar que el resultado es correcto, podemos hacer la conversión a la inversa utilizando el TFN, así pues:

$1101{,}0101_{(2} = 1 * 2^3 + 1 * 2^2 + 1 * 2^0 + 1 * 2^{-2} + 1 * 2^{-4} = 8 + 4 + 1 + 0{,}25 + 0{,}0625 = 13{,}3125$

1.4.4.3.- Conversión directa: octal a binario y viceversa.

a) Octal a binario. 8 → 2.

Puesto que las bases de los Sistemas Binario y Octal son potencias de 2, es decir, $2 = 2^1$ y $8 = 2^3$, las conversiones de octal a binario y viceversa se pueden realizar de forma directa. Para ello, hay que conocer la correspondencia de dígitos que existe entre ambas bases:

Octal	0	1	2	3	4	5	6	7
Binario	000	001	010	011	100	101	110	111

La conversión de los octales a binarios se hace reemplazando cada dígito octal por los 3 bits equivalentes.

NOTA: Aunque el dígito tenga suficiente con 2 bits para ser representado (dígito<4), ha de emplearse 3 bits.

Ejemplo: Para convertir el número octal $16{,}51_{(8}$ a binario, tomaremos el equivalente binario de cada uno de sus dígitos **con 3 bits**:

$1_{(8} = 001_{(2}$

$6_{(8} = 110_{(2}$

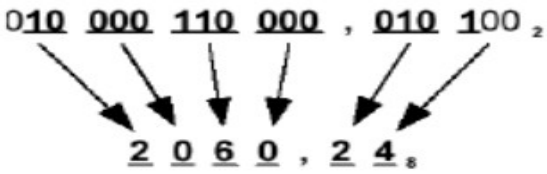
$5_{(8} = 101_{(2}$

$1_{(8} = 001_{(2}$

Por lo tanto: $16{,}51_{(8} = 1110{,}101001_{(2}$

b) Binario a octal. 2 → 8.

La conversión de números binarios a octales se debe hacer corresponder grupos de 3 bits con sus equivalentes en octal. De forma que;



Los ceros a la izquierda de la parte entera o a la derecha de la parte fraccionaria se desprecian. Así pues obtenemos el resultado que ya sabíamos, que $16{,}51_{(8} = 1100{,}101001_{(2}$

1.4.4.4.- Conversión directa: hexadecimal a binario y viceversa.

Debido a que la base del Sistema Hexadecimal también es una potencia de dos, $16 = 2^4$, se puede utilizar la tabla de correspondencias entre los dígitos de los Sistemas Hexadecimal y Binario. Observa la siguiente tabla de correspondencias entre los dígitos del Sistema Hexadecimal y Binario:

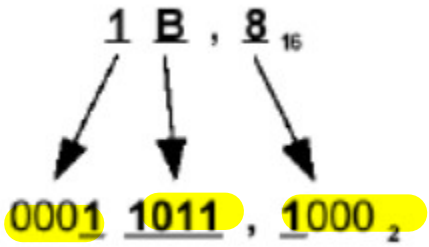
Hexadecimal	0	1	2	3	4	5	6	7
Binario	0000	0001	0010	0011	0100	0101	0110	0111

Hexadecimal	8	9	A	B	C	D	E	F
Binario	1000	1001	1010	1011	1100	1101	1110	1111

a) Hexadecimal a binario. 16 → 2

La conversión de números hexadecimales a binarios se hace reemplazando cada dígito hexadecimal por los 4 bits equivalentes.

Ejemplo: Para convertir de forma directa el 1B,816 a base 2, hay que hacer corresponder cada uno de sus dígitos con sus cuatro equivalentes en binario:



Los ceros a la izquierda de la parte entera o a la derecha de la parte fraccionaria se desprecian,

quedando: 1B,816 = 11011,12.

b) Binario a hexadecimal. 2 → 16

En la conversión de los binarios a hexadecimales se debe hacer corresponder grupos de cuatro bits con sus dígitos equivalentes en hexadecimal. De tal manera que el binario 1 0000 0000 1111 , 112 en hexadecimal sería:

$1111_{(2} = F_{(16}$
$1100_{(2} = C_{(16}$

Por lo tanto, 1 0000 0000 1111 $1100_{(2} = 100FC_{(16}$

1.4.5.- Aritmética binaria.

Todas las operaciones matemáticas (sumas, restas, multiplicaciones, divisiones, etc) que realiza la computadoras están basadas en la aritmética binaria. La aritmética binaria es muy similar a la decimal ya que sólo se manejan dos números, el 0 y el 1.

a) Suma.

Para realizar una suma hay que tener en cuenta la siguiente tabla:

Suma binaria	
0 + 0 = 0	1 + 0 = 1
0 + 1 = 1	1 + 1 = 0 y acarreo 1

Ejemplo: sumar $10010_{(2} + 110_{(2}$ se puede escribir:

		1	1			← Acarreos
	1	0	0	1	0	← 1º Sumando
+			1	1	0	← 2º Sumando
	1	1	0	0	0	← Resultado

b) Resta.

Para realizar una resta binaria hay que tener en cuenta la siguiente tabla:

Resta binaria	
0 – 0 = 0	1 - 0 = 1
0 -1 = 1 y acarreo 1	1 - 1 = 0

Ej: Resta los números decimales 63 y 42 en binario.

En primer lugar, convertimos ambos números a binario mediante los métodos ya conocidos:

$63 = 111111_{(2}$

$42 = 101010_{(2}$

Una vez convertidos, procedemos a restarlos:

$$\begin{array}{cccccc} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ \hline 0 & 1 & 0 & 1 & 0 & 1 \end{array}$$

Si fuese necesario, se vuelve a convertir el resultado a decimal:

$10101_{(2} = 1*2^4+1*2^2+1*2^0 = 16 + 4 +1 = 21$

Como podemos ver, coincide con el resultado en decimal, ya que 63 – 42 = 21.

c) Multiplicación.

Para realizar una multiplicación binaria hay que tener en cuenta la siguiente tabla:

Multiplicación binaria	
0 * 0 = 0	1 * 0 = 0
0 * 1 = 0	1 * 1 = 1

Ej: Multiplica los números 6 y 2 en binario.

En primer lugar, convertimos 6 y 2 a binario:

$6 = 110_{(2}$

$2 = 10_{(2}$

$$\begin{array}{cccc} & 1 & 0 & 1 \\ & & 1 & 0 \\ \hline & 0 & 0 & 0 \\ 1 & 0 & 1 & \\ \hline 1 & 0 & 1 & 0 \end{array}$$

De nuevo, podemos convertir el resultado a decimal mediante el TFN:

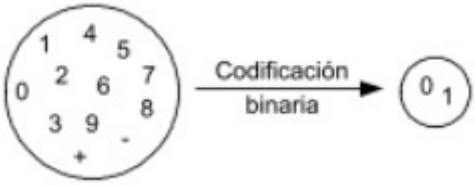
$1010 = 1*2^3 + 1*2^1 = 8 +4 = 12.$

Como podemos ver, coincide con el resultado en decimal, ya que 6 * 2 = 12.

1.4.6.-Representación de números enteros.

Para poder trabajar con números negativos en binario es necesario utilizar otro sistema de representación. Las cuatro formas habituales que se utilizan para estas representaciones son:

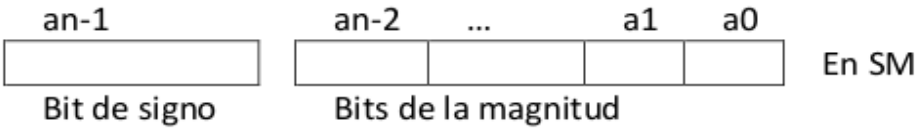
- Módulo y signo.
- Complemento a 1
- Complemento a 2.
- Exceso a 2 elevado a N – 1 (exceso $2^N – 1$)



Rango de representación: es el conjunto de números representables entre un valor mínimo y un valor máximo, ambos inclusive. El factor que más influye en el tamaño de un rango de representación es n, es decir, el nº de bits que se destinen para representar a los números.

1.4.6.1.- Signo – magnitud.

Se representan números positivos y negativos. En este sistema, el bit más significativo (situado más a la izquierda) representa el signo, y su valor será 0 para el signo + y 1, para el -. El resto de bits (n-1) representan a la magnitud o módulo.



En Signo Magnitud, el rango de representación que puede tomar un cierto número x, viene dado por la expresión:

$$(-2^{n-1} + 1)_{10} \leq x \leq (2^{n-1} - 1)_{10}$$

Ejemplo: En Signo Magnitud, para n = 8, el rango de representación es:

$$(-2^{8-1} + 1)_{10} \leq x \leq (2^{8-1} - 1)_{10}$$
$$(-2^{7} + 1)_{10} \leq x \leq (2^{7} - 1)_{10}$$
$$(-128 + 1)_{10} \leq x \leq (128 - 1)_{10}$$
$$-127_{10} \leq x \leq 127_{10}$$

O sea, se pueden representar 2^8 -1= 255 números enteros que van desde el -127 hasta el +127.

La ventaja de este sistema es que posee un rango simétrico (igual cantidad de números positivos y negativos).

Ejemplo: En Signo Magnitud, para n = 4, ¿cómo se escriben los números 7 y -7?

	a3	a2	a1	a0	
7_{10} =	0	1	1	1	En SM
	Signo	Magnitud			
	a3	a2	a1	a0	
-7_{10} =	1	1	1	1	En SM
	Signo	Magnitud			

Las desventajas de este Sistema son:

a) Puede haber desbordamiento (overflow).

b) El número cero va a tener dos representaciones diferentes.

$$0_{10} = 0000000_{SM} = 10000000_{SM}$$

c) Operaciones aritméticas más complejas. A la hora de sumar y restar dos números en Signo y Magnitud, tendremos dos casos:

· **Caso 1**, signos diferentes: Se resta el mayor al menor y el signo será el del mayor.

4 – 3 → 0100 + 1011 → + (100 - 011) = 0001 → +1

- 4 + 3 → 1100 + 0011 → - (100 – 011) = 1001 → -1

· **Caso 2**, signos iguales: Se mantiene el signo y se suman los valores absolutos.

4 + 5 → 0100 + 0101 → + (100 + 101) = 01001 Desbordamiento.

-2 – 3 → 1010 + 1011 → - (010 + 011) = 1101 → -5

1.4.6.2.- Complemento a 1.

Este sistema de representación utiliza el bit situado más a la izquierda para el signo, correspondiendo el 0 para el signo + y el 1 para el signo -. Para los números positivos, los N - 1 bits de la derecha representan el módulo, y el negativo de un número se obtiene complementando todos sus dígitos (cambiando ceros por unos y viceversa), incluido el bit de signo.

Ejemplo: En Complemento a 1, para n = 3, ¿cómo se escribe el número 3 y -3?

	a2	a1	a0	
3_{10} =	0	1	1	En C1
	a2	a1	a0	
-3_{10} =	1	0	0	En C1

Ejercicio: En Complemento a 1, para n = 8, ¿cómo se escribe el número 10 y -10?

	a7	a6	a5	a4	a3	a2	a1	a0	
10_{10} =	0	0	0	0	1	0	1	0	En C1

	a7	a6	a5	a4	a3	a2	a1	a0	
-10_{10} =	1	1	1	1	0	1	0	1	En C1

Las desventajas de este sistema son:

- Desbordamientos.
- Dos representaciones para el número cero.

$$0_{10} = 00000000000_{SM} = 1111111111111111_{SM}$$

En Complemento a 1, el rango de representación es el mismo que en Signo Magnitud:

$$\boxed{(-2^{n-1} + 1)_{10} \leq x \leq (2^{n-1} - 1)_{10}}$$

Ejemplo: En Complemento a 1, para n = 16, el rango de representación es:

n=8
-127 =< x =<127

$$(-2^{16-1} + 1)_{10} \leq x \leq (2^{16-1} - 1)_{10}$$
$$(-2^{15} + 1)_{10} \leq x \leq (2^{15} - 1)_{10}$$
$$(-32768 + 1)_{10} \leq x \leq (32768 - 1)_{10}$$

1.4.6.3.- Complemento a 2.

Los números positivos se representan igual que en binario puro, mientras que para calcular los números negativos hacemos el C1 y le sumamos 1, despreciando el último acarreo si existe.

En Complemento a 2, el rango de representación es:

$$\boxed{(-2^{n-1})_{10} \leq x \leq (2^{n-1} - 1)_{10}}$$

Ejemplo: En Complemento a 2, para n = 16, el rango de representación es:

$$(-2^{16-1})_{10} \leq x \leq (2^{16-1} - 1)_{10}$$
$$(-2^{15})_{10} \leq x \leq (2^{15} - 1)_{10}$$
$$(-32768)_{10} \leq x \leq (32768 - 1)_{10}$$

n=8
-128 =< x =<127
5 : 0000 0101
-5: 1111 1010+1
-5: 1111 1011

Las desventajas de este Sistema son:

· Desbordamientos.

· Asimetría.

Ejemplo: En Complemento a 2, para n = 8, ¿cómo se escriben los números 10 y -10?

· 10:

Es igual que en Complemento a 1, Signo Magnitud y Binario Puro:

10_{10} =	0	0	0	0	1	0	1	0	En C2

· -10:

Primero se complementa cambiando ceros por unos y viceversa incluido el bit de signo:

-10_{10} =	1	1	1	1	0	1	0	1	En C1

Una vez complementado a 1, se suma 1, y el resultado es el esperado:

-10_{10} =	1	1	1	1	0	1	1	0	En C2

1.4.7.- Representación alfanumérica.

Para finalizar, recordar que los ordenadores no sólo necesitan representar números, sino también letras y otros símbolos. Para ello, es preciso el uso de códigos alfanuméricos, capaces de representar:

- Números
- Caracteres alfabéticos
- Símbolos
- Instrucciones necesarias para la transferencia de información.

Para representar caracteres, se asigna a cada carácter un octeto o byte. Los tipos de codificación binaria más habituales y que ya hemos estudiado son:

- ASCII
- ASCII extendido
- Unicode.